Supervision centralisée

BUT informatique 3A, parcours DACS, mini-rapport

Binôme : Noa Craincourt / Camil Taleb
Novembre 2026

Architecture

L'ensemble est décrit dans un seul `compose.yaml` qui lance quatre conteneurs. Le premier, `agent-hote`, audite notre machine avec les options du projet précédent et répond sur le port 9100. Le deuxième, `agent-conteneur`, audite son propre conteneur, sans aucun accès à l'hôte, et répond sur le port 9101. Prometheus collecte les deux agents toutes les 15 secondes, et Grafana affiche les données. Le tout se lance avec `docker compose up -d --build`.

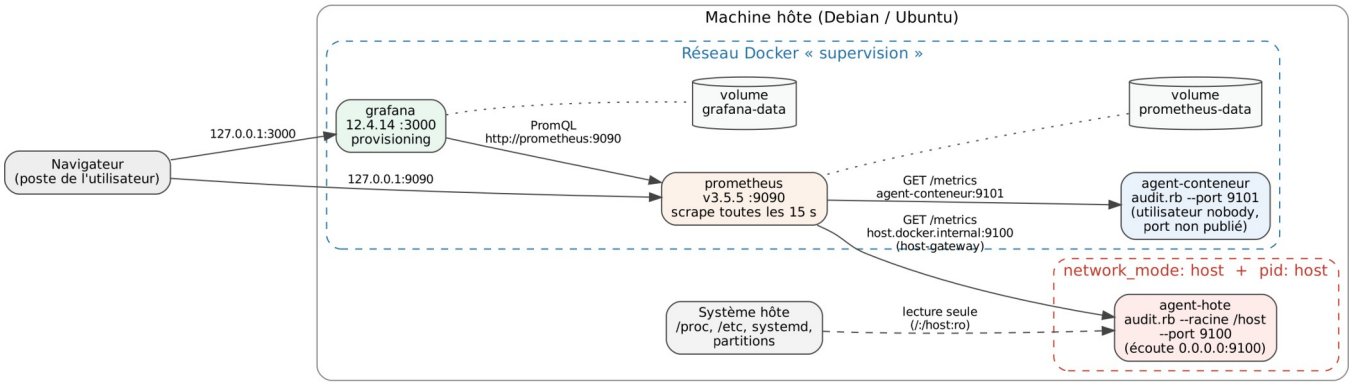


Figure 1 : architecture de la solution

1 Prise en main de Prometheus

Qu'est-ce qu'une série temporelle et comment est-elle identifiée (nom de métrique et labels) ? Une série temporelle est une suite de mesures numériques relevées au fil du temps. Dans Prometheus, elle est identifiée de manière unique par un **nom de métrique** (ex: `audit_load_average`) auquel on associe des **étiquettes (labels)** sous forme de paires clé/valeur. Cela permet de classer et filtrer précisément la donnée. Si on prend un exemple : `audit_load_average{period="1m"}`

- audit_load_average est le nom de la métrique.
- period="1m" est le label.
  - period est la clé.
  - 1m est la valeur.

Quel est le fonctionnement du modèle pull, par opposition au modèle push ? Le modèle **pull**

signifie que c'est Prometheus qui va de lui-même interroger les applications pour récuperer les données dont il a besoin. Le modèle **push** est l'inverse, c'est les applications qui envoient les données a Prometheus.

Quelle est la différence entre les types "counter" et "gauge" ?

- **Counter (compteur) :** C'est une valeur qui ne peut que croître (ou être réinitialisée à zéro si il y a par exemple un redémarage). Elle est utilisée pour des cumuls, comme le temps total d'utilisation du processeur depuis le démarrage.
- **Gauge (jauge) :** C'est une valeur instantanée qui peut monter ou descendre librement, comme la mémoire utilisée, la charge moyenne du système ou l'espace disque disponible.

A quoi servent les sélecteurs et la fonction `rate()` en PromQL ?

- **Les sélecteurs :** Ils s'écrivent entre accolades `{}` et permettent de filtrer les métriques que l'on veut afficher dans Grafana en ciblant des labels précis. Il y a deux types de sélecteurs :
  - Sélecteur intantané :
    - Exemple : http_requests_total{env="production", method="GET"} : Il cherche la métrique http_requests_total et filtre pour ne garder que la production et la méthode GET. Il renvoie une seule valeur par série
  - Sélecteur de plage :
    - Exemple : http_requests_total{env="production"}[5m] : Il va chercher toutes les valeurs enregistrées par cette métrique au cours des 5 dernières minutes.
- **La fonction `rate()` :** Elle s'utilise uniquement sur un *counter* pour calculer sa vitesse d'augmentation moyenne par seconde sur une période donnée. Par exemple, si on veut regarder le nombre de requêtes par seconde sur un site internet, on peut utiliser rate par exemple sur les 5 dernières minutes, et elle va prendre le nombre total de requêtes de ces 5 dernières minutes et va les diviser par le nombre de secondes pour avoir le nombre de requête par seconde. Cela peut permettre de voir a quels moments de la journée le site est le plus surchargé.

2 Adaptation de l'agent d'audit

Notre script faisait un audit puis s'arrêtait. Prometheus fonctionne en *pull* : il lui faut un processus permanent qui répond à `GET /metrics`. Nous avons donc ajouté un format de sortie `prometheus` (`ruby audit.rb -f prometheus`) et une option `--port` qui lance un serveur HTTP. À chaque requête sur `/metrics`, le serveur relance l'audit (fonction `generer_audit_global`) et renvoie les métriques au format texte. Toute autre adresse renvoie une 404.

Pour le serveur, nous avons choisi `TCPServer`, de la bibliothèque standard. Il ne demande rien à installer et tient en une vingtaine de lignes. Les autres solutions ajoutaient une dépendance : WEBrick n'est plus fourni depuis Ruby 3.0, la gem `prometheus-client` doit être installée, et node_exporter ou json_exporter demandent un conteneur de plus. Pour un seul client qui passe toutes les 15 secondes, `TCPServer` suffit.

Métriques exposées

Métrique	Type	Labels	Source
`audit_load_average`	gauge	`period` (1m, 5m, 15m)	`/proc/loadavg`
`audit_uptime_seconds_total`	counter	aucun	`/proc/uptime`
`audit_memory_bytes`	gauge	`state` (total, used, available)	`/proc/meminfo`
`audit_disk_bytes`	gauge	`mount`, `state` (total, used, available)	`df -P`
`audit_service_active`	gauge	`service`	`systemctl is-active`
`audit_cpu_ticks_total`	counter	`mode` (user, system, idle, iowait)	`/proc/stat`

Extrait de notre sortie `/metrics` :

```
audit_load_average{period="1m"} 0.23
audit_memory_bytes{state="total"} 16625950720
audit_memory_bytes{state="used"} 4561227776
audit_disk_bytes{mount="/host", state="used"} 74279911424
audit_service_active{service="ssh"} 0
audit_service_active{service="cron"} 1
audit_cpu_ticks_total{mode="user"} 67877.0
audit_cpu_ticks_total{mode="idle"} 2160362.0
```

Nos choix

Nous avons suivi les conventions de Prometheus : un préfixe commun `audit_`, l'unité en suffixe (`_bytes`, `_seconds`) et `_total` pour les compteurs. Les kilo-octets donnés par `/proc/meminfo` et `df` sont convertis en octets. Pour la mémoire et les disques, nous utilisons une seule métrique avec un label `state` plutôt que trois métriques séparées, ce qui permet de calculer un pourcentage directement avec `ignoring(state)`. Le statut d'un service est un texte : il devient 1 quand `systemctl is-active` répond `active`, et 0 sinon.

Le CPU nous a demandé plus de réflexion. `/proc/stat` ne donne pas une utilisation instantanée, mais le temps cumulé passé dans chaque mode depuis le démarrage, en ticks (1/100 s chez nous, d'après `getconf CLK_TCK`). Nous l'exposons donc en compteur et c'est Prometheus qui calcule l'utilisation :

```
100 * (1 - sum by (machine) (rate(audit_cpu_ticks_total{mode="idle"}[1m]))
         / sum by (machine) (rate(audit_cpu_ticks_total[1m])))
```

On divise des ticks par des ticks, donc pas besoin de connaître `CLK_TCK`. La métrique s'appelle `_ticks_total` et pas `_seconds_total`, puisque sa valeur n'est pas en secondes.

Nous n'avons pas exporté les processus, les connexions, les utilisateurs ni les interfaces réseau. Un label `pid` ou `port` créerait une nouvelle série temporelle à chaque valeur, soit des milliers de séries inutiles. La distribution et le noyau sont du texte qui ne change pas : une courbe n'aurait pas d'intérêt.

L'audit répond en 40 à 120 ms selon l'agent, loin du `scrape_timeout` de 10 s. `promtool check metrics` ne renvoie aucune erreur.

3 Déploiement des agents

Les deux agents utilisent la même image, avec une configuration différente :

	agent-hote	**agent-conteneur**
Cible	La machine hôte	Son propre conteneur
Options	`pid: host`, `network_mode: host`, `/:/host:ro`	Aucune
Arguments	`--racine /host --port 9100 --services ssh,cron,docker`	`--port 9101 --services cron,ssh`
Droits	root + `SYS_PTRACE`	utilisateur `nobody`, aucune capacité

L'agent hôte est sur le réseau de l'hôte, donc Prometheus ne peut pas le joindre par son nom de service. Nous passons par l'adresse de l'hôte grâce à `extra_hosts: "host.docker.internal:host-gateway"`. Comme il occupe directement le port 9100 de l'hôte, il entrerait en conflit avec un node_exporter déjà installé.

En comparant les deux agents, nous avons remarqué que l'agent conteneur affiche presque les mêmes valeurs que l'hôte pour le CPU, la mémoire, la charge et l'uptime. C'est normal : les fichiers de `/proc` ne sont pas isolés par les namespaces. Malgré `mem_limit: 256m`, il affiche les 15,5 Go de l'hôte (figure 5). Les vraies valeurs du conteneur sont dans les cgroups, que le script ne lit pas. Ses services sont tous à 0, puisqu'il n'y a pas de systemd dans un conteneur.

Sécurité. Les métriques ne sont pas authentifiées et l'agent hôte écoute sur toutes les interfaces : n'importe qui sur le réseau local peut les lire. Elles révèlent les services actifs (SSH par exemple), l'uptime, qui donne une idée des mises à jour appliquées, et la taille des disques. Nous n'avons publié aucun port pour l'agent conteneur, et Prometheus et Grafana ne sont accessibles que sur `127.0.0.1`. Pour l'agent hôte, il faut un pare-feu qui n'autorise le port 9100 que depuis le réseau Docker.

4 Déploiement de Prometheus

`prometheus.yml` déclare un job `audit` avec deux cibles : `host.docker.internal:9100` et `agent-conteneur:9101`. Nous utilisons le nom de service, car `localhost` désignerait le conteneur Prometheus lui-même. Chaque cible reçoit un label `machine` (`hote` ou `agent-conteneur`) pour distinguer les machines. Nous avons réduit le `scrape_interval` à 15 s pour avoir des courbes plus précises. L'image est fixée en `v3.5.5` (version LTS), et l'historique est conservé dans un volume monté sur `/prometheus`. `promtool check config` valide la configuration.

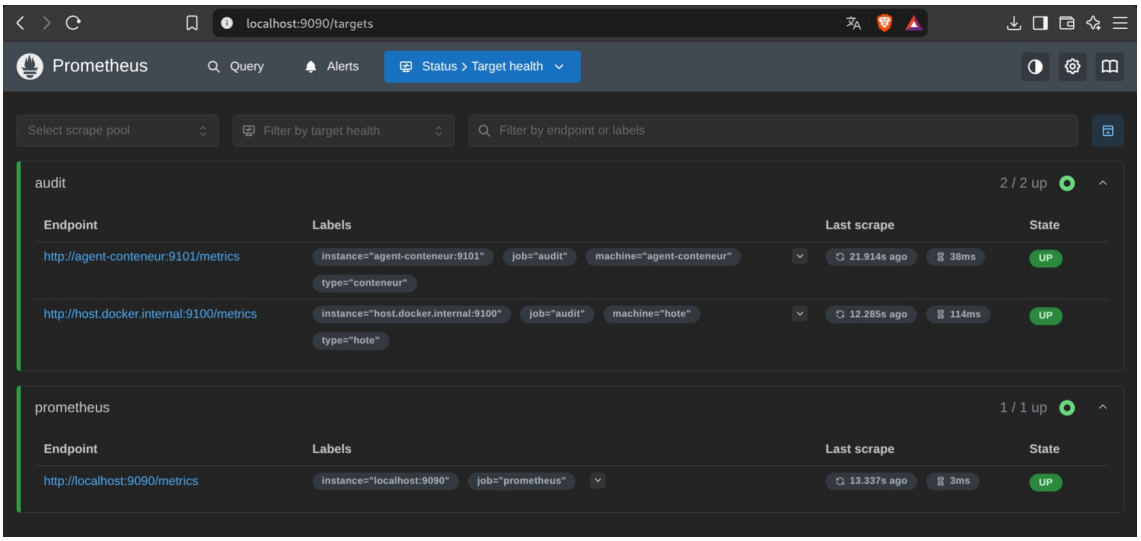


Figure 2 : cibles collectées par Prometheus

La page *Target health* montre les trois cibles `UP` avec nos labels (figure 2). La figure 3 montre la requête `audit_load_average` : on y voit le pic de charge au lancement de la stack, et la moyenne sur 15 minutes qui réagit plus lentement que celle sur 1 minute.

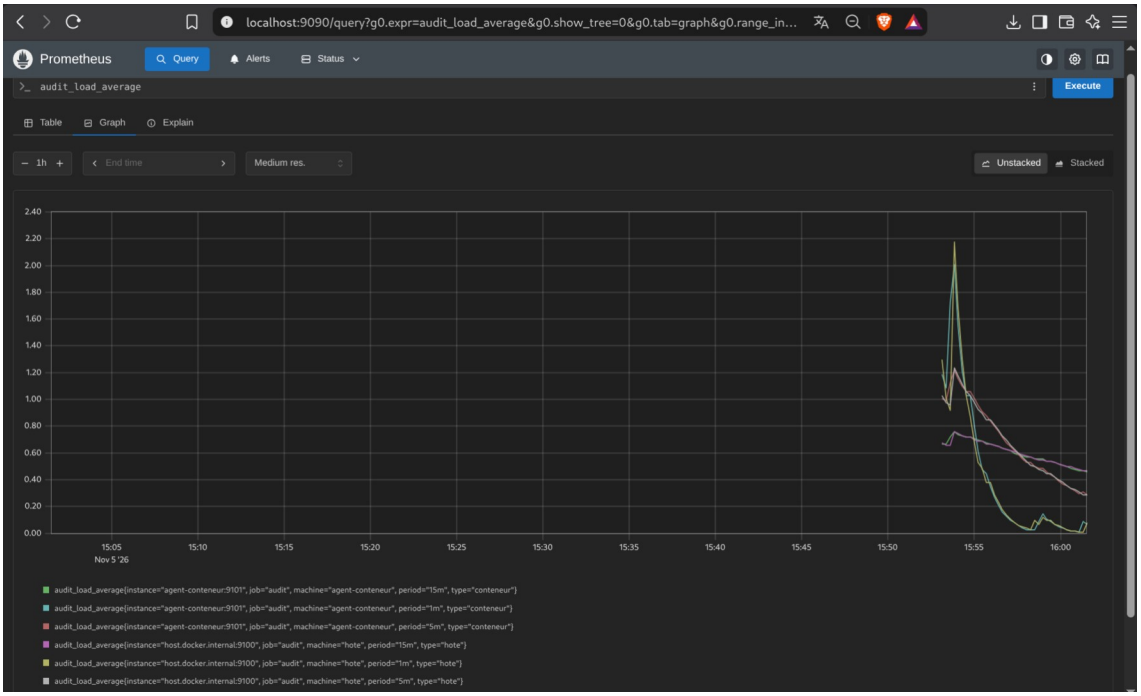


Figure 3 : requête audit_load_average

5 Déploiement de Grafana

La source de données pointe vers `http://prometheus:9090`, l'adresse de Prometheus vue

depuis le conteneur Grafana. Le mot de passe administrateur est lu dans un secret Docker (`GF_SECURITY_ADMIN_PASSWORD__FILE`), et le fichier qui le contient est exclu de Git.

Le tableau de bord utilise une variable `machine`, issue du label ajouté dans Prometheus (`label_values(up{job="audit"}, machine)`). Toutes les requêtes filtrent avec `machine=~"$machine"`, ce qui permet d'afficher une machine ou de les comparer avec le même tableau de bord.

Information	Panneau	Requête (simplifiée)
Agents joignables	Stat	`up{job="audit"}`
CPU (%)	Courbe	`100 * (1 - rate(idle) / rate(total))`
Charge moyenne	Courbe	`audit_load_average`
Mémoire (%)	Jauge	`used / ignoring(state) total`
Disques (%)	Jauge	`used / ignoring(state) (used + available)`
Services	Stat et state timeline	`audit_service_active`

Pour les disques, nous calculons comme `df` : utilisé / (utilisé + disponible), car l'espace réservé à root n'apparaît dans aucun des deux. Les captures nous ont aussi montré un problème : sur Ubuntu, chaque snap est monté depuis un `/dev/loop` et apparaît comme un disque plein à 100 %. Nous les avons exclus avec `mount!~".*/snap/.*"`.

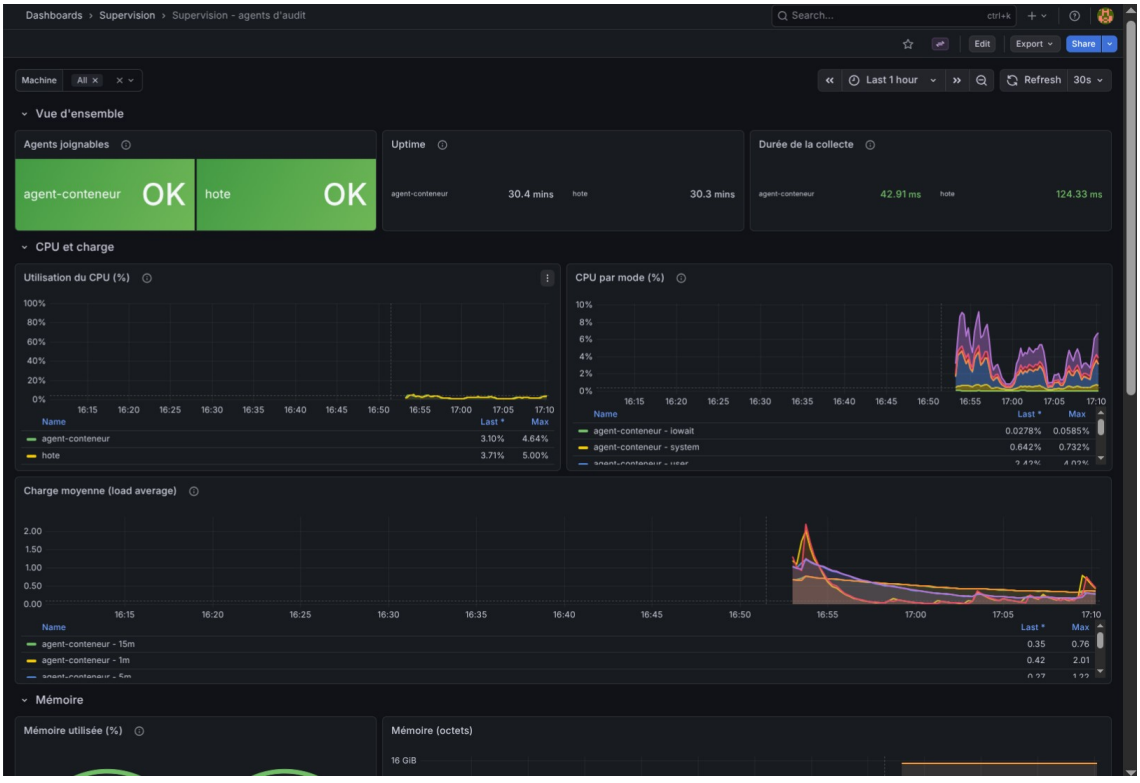


Figure 4 : tableau de bord (CPU, charge, mémoire)

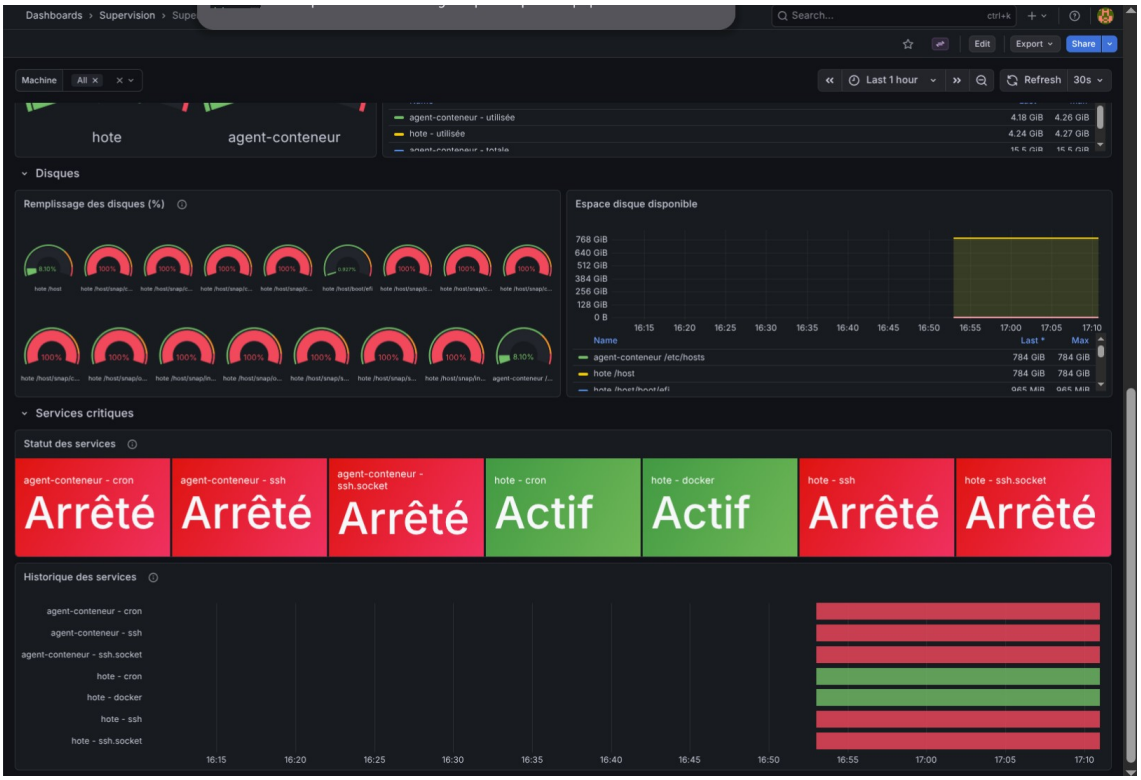


Figure 5 : tableau de bord (disques et services)

Sur la figure 5, `ssh` est en rouge sur l'hôte : il était arrêté sur notre machine, ce qui nous a servi d'exemple de service en panne. Le tableau de bord est exporté en JSON et versionné dans `grafana/dashboards/audit.json`.

Bonus : provisioning. La source de données et le tableau de bord sont décrits dans des fichiers montés dans `/etc/grafana/provisioning/`. Un `docker compose up` suffit donc à tout reconstruire, sans configuration manuelle. Nous avons fixé l'`uid` de la source de données pour que le JSON du tableau de bord la retrouve.

Limites et perspectives

Limites. `TCPServer` ne traite qu'une requête à la fois. L'audit complet est relancé à chaque collecte, y compris des commandes inutiles pour Prometheus (`ss`, `ps`, `ip`). Seuls 4 modes CPU sont exposés sur une dizaine. Un service absent et un service arrêté donnent tous les deux 0. L'agent conteneur affiche surtout les valeurs de l'hôte.

Passage à l'échelle. Les cibles sont écrites à la main. Avec beaucoup de machines, il faudrait de la découverte automatique (`file_sd_configs`, DNS), un agent déployé sur chaque machine (avec Ansible par exemple), et peut-être plusieurs Prometheus.

Sécurité. Les métriques circulent en clair. Prometheus gère TLS et l'authentification avec `--web.config.file`. Pour l'agent, il faudrait un reverse proxy. L'agent hôte tourne en root et voit

toute l'arborescence de l'hôte, ce qui est risqué en cas de faille.

Automatisation. Il faudrait ajouter des alertes avec Alertmanager (agent injoignable, disque plein, service arrêté) et vérifier les fichiers automatiquement avec `promtool` à chaque commit.

PS : le mot de passe Graphana est 123mdp*.